OBJECTS VS. POINTERS

Pointers to objects

- So far we've treated objects like ordinary variables and assumed that an object is created in the place where it is declared and destroyed when its declaration scope is exited.
- Objects may also exist as dynamically created and destroyed entities.
- In other words, **objects may appear on demand** – when they're needed – and vanish in the same way.

Pointers to objects

```
class Class {
public:
        Class(void) {
                cout << "Object constructed!" << endl;
        }
        ~Class(void) {
                cout << "Object destructed!" << endl;
}
};
int main(void) {
        Class *ptr;

        ptr = new Class();
        delete ptr;
        return 0;
        }
```

Output:

Object constructed!

Object destructed!

Pointers to objects

- We've created one object of that class using the ***new*** keyword.
- Note that **we can omit** the empty parentheses after the *Class* name – in either case, the parameter-less constructor will be activated.
- The object is destroyed using the ***delete*** keyword.
- The process of destroying the object begins with the implicit invocation of its destructor.

Pointers to fields

- All the variables, including objects, brought to life in the "ordinary" way live in **a separate area of memory called the *stack***.
- It's a memory region dedicated to **storing all automatic entities**.
- The stack grows when new automatic variables are created and shrinks when the variables are no longer needed.
- Note that this process is beyond your control. **You cannot affect** the way in which the stack changes.

Pointers to fields

- The entities created “on demand” (by the *new* keyword) are created in **a specific memory region usually called a *heap***.
- In contrary to the stack, **the heap is fully under your control**. You decide how many variables, arrays, objects, etc. will occupy the heap and it’s up to you when these entities end their lives.
- The object being stored in the heap must be accessed in a way that resembles the access to the dynamically allocated structures.
- You mustn’t use the ordinary “dotted” notation as there’s no structure (object) which can play the role of the left argument of the “.” operator unless you dereference the pointer.
  - **You need to use the “arrow” (->) operator instead**.

Pointers to fields

- The general rule says that:

  if *S* is a **structure** or **class** and *S* has a **component** named *C* and if *p* is a pointer to a structure of type *S,* then the *C* component may be accessed in the two following ways:

  - `(*p).C;` // *p* is explicitly dereferenced in order to access the *C* component
  - `p->C;` // *p* is implicitly dereferenced in order to access the *C* component

Note: *ptr.value = 0; // Produce error, as "**.**" has higher precedence than " * "

Pointers to fields

- We've added one field to the *Class*.
  - It's declared in the public part of the class so you can access it freely from outside the class.
  - There's one catch – you have to use the "->" operator.
  - You can use * with . Instead of -> operator.

```
  ptr->value = 0;
 (*ptr).value = 0;
```

  - Look at the parentheses around the argument of the ++ operator.
    - Do we really need them? What would happen when you removed them?

Pointers to fields

```
class Class {
public:
	Class(void) {
		cout << "Object constructed!" << endl;
	}
	~Class(void) {
		cout << "Object destructed!" << endl;
	}
	int value;
};
int main(void) {
	Class *ptr;

	ptr = new Class;
	ptr -> value = 0;
	cout << ++(ptr -> value) << endl;
	delete ptr;
	return 0;
}
```

Output:

Object constructed!
1
Object destructed!

Pointers to functions

- Member functions invoked for an object accessed through the pointer have to be accessed using the arrow operator, too.

```
class Class {
public:
	Class(void) {
		cout << "Object constructed!" << endl;
	}
	~Class(void) {
		cout << "Object destructed!" << endl;
	}
	void IncAndPrint(void) {
		cout << "value = " << ++value << endl;
	}
	int value;
};
int main(void) {
	Class *ptr;

	ptr = new Class;
	ptr -> value = 1;
	ptr -> IncAndPrint();
	delete ptr;
	return 0;
}
```

Output:
Object constructed!
Value = 2
Object destructed!

Selecting the constructor

- If a class has more than one constructor, one of them may be chosen during object creation.
  - **This is done by specifying the form of the parameter list associated with the class name.**
- The list should be unambiguously compatible with one of the available class constructors.
- We've modified our program again.
  - There are already two constructors.
  - We've created two new objects inside the main functions. They differ in the constructor used to build each of the objects. In effect, their value fields have different values assigned to them.

Selecting the constructor

```
class Class {
public:
        Class(void) { cout << "Object constructed (#1)" << endl; }
        Class(int v) { value = v; cout << "Object constructed (#2)" << endl; }
        ~Class(void) { cout << "Object destructed! val = " << value << endl; }
        void IncAndPrint(void) {
                cout << "value = " << ++value << endl;
        }
        int value;
};
int main(void) {
        Class *ptr1, *ptr2;
        ptr1 = new Class;
        ptr2 = new Class(2);
        ptr1 -> value = 1;
        ptr1 -> IncAndPrint();
        ptr2 -> IncAndPrint();
        delete ptr2;
        delete ptr1;
        return 0;
}
```

Output:

```
Object constructed (#1)
Object constructed (#2)
value = 2
value = 3
Object destructed! val = 3
Object destructed! val = 2
```

Classes and Pointers: Some Peculiarities

```
class pointerDataClass
{
public:
    .
    .
    .
private:
    int x;
    int lenP;
    int *p;
};
pointerDataClass objectOne;
pointerDataClass objectTwo;
```

There are three things to take care when the class uses pointer members:

1. Destructor to delete the dynamic memory
2. Overload the assignment operator
3. Override the Copy constructor

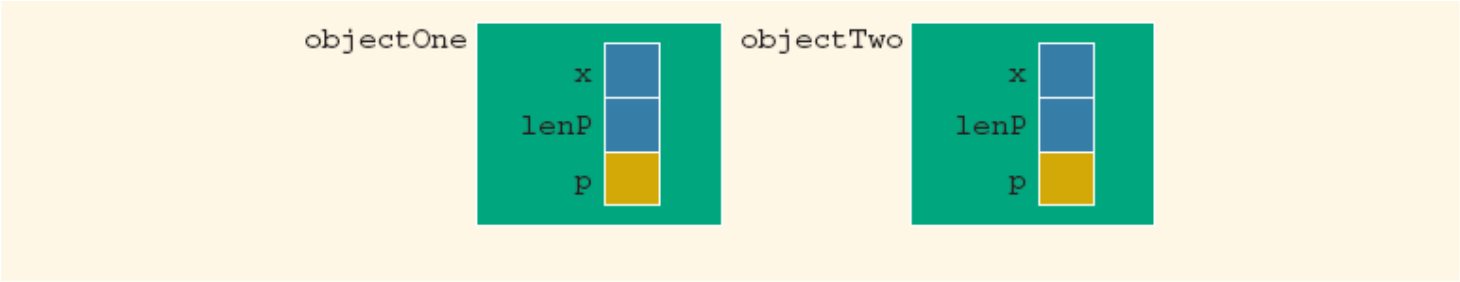


FIGURE 14-21 Objects objectOne and objectTwo

Memory leaks

- Many of the objects are allocated memory that they need for their operation. This memory should be released when the object finishes its activity and the best way to do this is to do the cleaning automatically.
- Failure to clean the memory will cause a “**memory leaking**”, where the unused (but still allocated!) memory grows in size, affecting system performance.
- Take a look at the example on the following slides.
- The *Class* class has only one constructor, which is responsible for allocating memory of the size specified by its parameter value. The object of this class is created as a local variable inside the *MakeALeak()* functions.

Memory leaks

```
#include <iostream>
using namespace std;
class Class {
public:
        Class(int val) {
                value = new int[val];
                cout << "Allocation (" << val << ") done." << endl;
        }
        int *value;
};
void MakeALeak(void) {
        Class object(1000);
}
int main(void) {
        MakeALeak();
        return 0;
}
```

Memory leaks

- the constructor **explicitly allocates another part of the memory**

- The *object* variable is an example of an "**automatic variable**". This means that the variable automatically finishes its life when the execution of the function containing the variable's declaration ends.
- We can expect that a return from the *MakeALeak()* functions will cause the following action:
  - **the memory allocated to the object itself is freed**(this is done implicitly).

- Unfortunately, the memory explicitly allocated by the constructor **remains allocated**. To make matters worse, we've lost the only pointer that held the address of that memory (it was stored by the *value* field, but the object containing this field doesn't exist anymore).
- The fairly large portion of memory has leaked.

Destructor

- If `objectOne` goes out of scope, the member variables of `objectOne` are destroyed
  - The memory space of the dynamic array would stay marked as allocated, even though it cannot be accessed

- Solution:
  - Put the necessary code in the destructor to ensure that when `objectOne` goes out of scope, the memory of the array is deallocated

```
pointerDataClass::~pointerDataClass()
{
    delete [] p;
}
```

Destructors

```
#include <iostream>
using namespace std;
class Class {
public:
        Class(int val) {
                value = new int[val];
                cout << "Allocation (" << val << ") done." << endl;
        }
        ~Class(void) {
                delete [ ] value;
                cout << "Deletion done." << endl;
        }
        int *value;
};
void MakeALeak(void) {
        Class object(1000);
}
int main(void) {
        MakeALeak();
        return 0;
}
```

Copy constructor

- There is a special kind of constructor intended to **copy one object into another**.
- They are referred to as **copy constructors** and are **implicitly invoked** when a declaration of an object is followed by an initiator.
- Constructors of this kind have **one parameter referenced to an object of the same class** and are used to copy all important data from the source object to the newly created object (or more precisely, to the object **currently being created**).

Copy constructors

```
class Class {
        int data;
public:
        Class(int value) : data(value) {}
        void increment(void) {        data++; }
        int value(void) { return data; }
};
int main(void) {
        Class o1(123);
        Class o2 = o1;
        Class o3(o2);

        o1.increment();
        cout << o1.value() << endl;
        cout << o2.value() << endl;
        cout << o3.value() << endl;

        return 0;
}
```

Output:
124
123
123

Copy constructors

- If the copy constructor doesn't exist within a particular class and the initiator is actually used during the declaration of an object, its content will be copied "field by field", as if the object had been cloned.

- The copy constructor will also be used **when the context requires a copy of a specific object**, e.g. when a particular object is transferred to a function as a value-passed actual parameter.

Copy constructors

```
class Dummy {
public:
	Dummy(int value) {}
	Dummy(Dummy &source) {
		cout << "Hi from the copy constructor!" << endl
;
	}
};
void DoSomething(Dummy ob) {
	cout << "I'm here!" << endl;
}
int main(void) {
	Dummy o1(123);
	DoSomething(o1);
	return 0;
}
```

Output:
Hi from the copy constructor!
I'm here!

Copy constructors

- The following example shows two different classes.
- The former does have a copy constructor, while the latter doesn't.
- Two objects are created for both of the classes while both of the second objects are created by copying the first ones.

Copy constructors

```
class Class1 {
public:
        Class1(int val) { this -> value = val; }
        Class1(Class1 const &source) { value = source.value + 100; }
        int value;
};
class Class2 {
public:
        Class2(int val) { this -> value = val; }
        int value;
};
int main(void) {
        Class1 object11(100), object12 = object11;
        Class2 object21(200), object22 = object21;
        cout << object12.value << endl;
        cout << object22.value << endl;
        return 0;
}
```

Output:
200
200

Copy Constructor (continued)

- Copy constructor automatically executes in three situations:
  - When an object is declared and initialized by using the value of another object
  - When, as a parameter, an object is passed by value
  - When the return value of a function is an object

Copy Constructor with pointer member

```
pointerDataClass objectThree(objectOne);
```

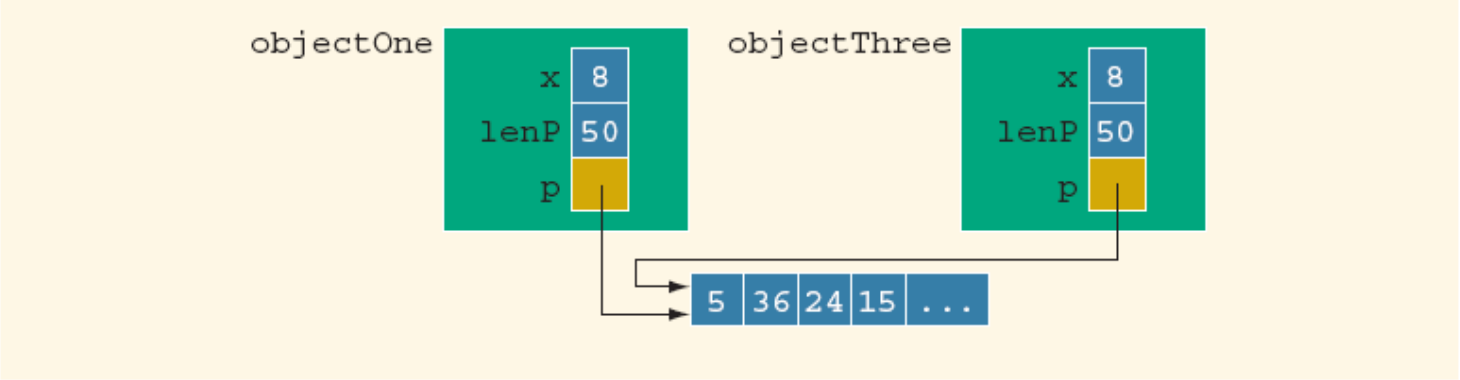


FIGURE 14-26 Objects `objectOne` and `objectThree`

- This initialization is called the default member-wise initialization
  - Initialization due to the constructor, called the **copy constructor** (provided by the compiler)

Copy Constructor (continued)

- Default initialization leads to **shallow copying** of data
- Similar problem occurs when passing objects by value:

```
void destroyList(pointerDataClass paramObject);
```

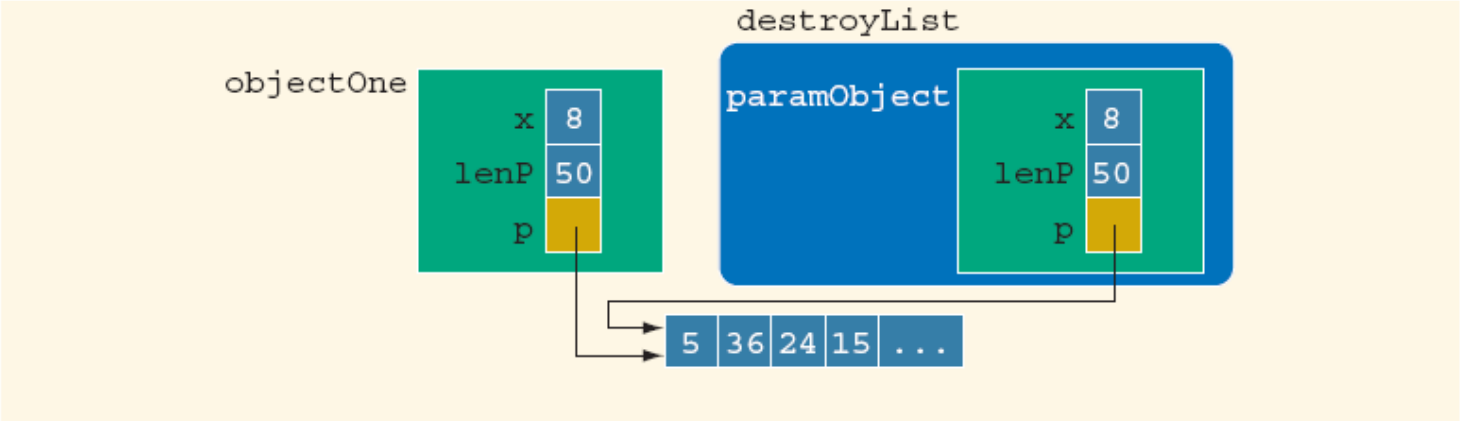


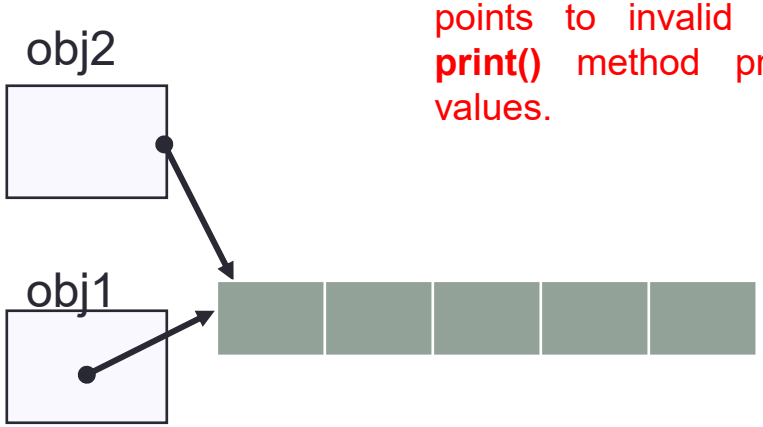
FIGURE 14-28 Pointer member variables of objects `objectOne` and `paramObject` pointing to the same array

- Solution: write a user copy constructor to do a **deep copy**

Default copy constructor

```
class List{
        int *data;
        int size;
public:
        List(int s)
        { size = s; data = new int[size]; }
        void fill()
        {
          for(int i = 0; i < size; i++)
            data[i] = i*i;
        }
        void print()
        {
            for(int i = 0; i < size; i++)
                cout<<data[i]<<"  ";
            cout<<endl;
        }
        ~List()
        {delete []data;}
};
void fun(List obj2)
{}
void main()
{       List obj1(5);
        obj1.fill();
        fun(obj1);
        obj1.print();}
```

When the **fun** method is called the **obj1** is **passed by value** to the **fun** method a **copy constructor** is invoked that creates a **copy** of **obj1,** which performs a shallow copy of the **data pointer**. So **obj1** and **ob2 both** share the same **data array in memory**. When the **fun** method finishes, the **destructor** for **obj2** is called that **deallocates** the memory for the **data array**. As a result, the **data pointer** in the **original obj1** points to invalid memory, thus the **print()** method prints the **garbage** values.

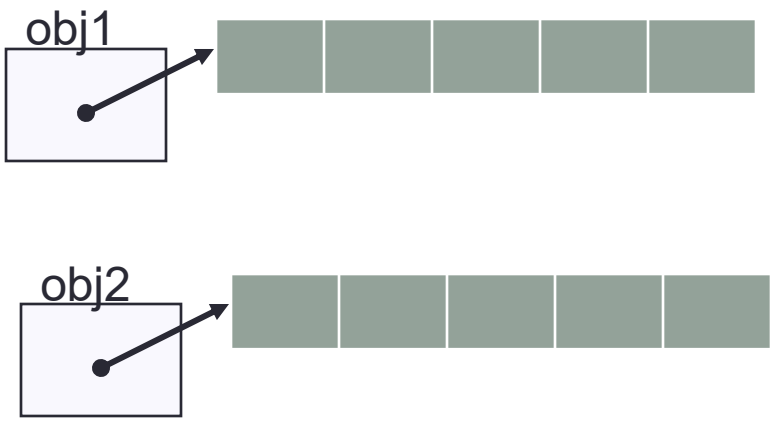


Output: Rubbish values

User defined copy constructor

```
class List{
        int *data;
        int size;
public:
        List(int s)
        { size = s;          data = new int[size];       }

        List(List &other)
        { size = other.size;
          data = new int[size];
          for(int i = 0; i < size; i++)
                data[i] = other.data[i];
        }
        void fill()
        {
          for(int i = 0; i < size; i++)
            data[i] = i*i;
        }
        void print()
        {
           for(int i = 0; i < size; i++)
              cout<<data[i]<<"  ";
            cout<<endl;
        }
        ~List()   {delete []data;}
};
void fun(List obj2)
{}
void main()
{
        List obj1(5);
        obj1.fill();
        fun(obj1);
        obj1.print();}
```



Output:

0 1 4 9 16

Arrays of pointers to objects (1)

- There's a class that implements a very simple array containing elements of type *int*.
- The **size of the array** is determined by the parameter's value passed to the constructor of the class.
- The class offers us two methods for accessing the array.
  - The first (named ***`Get`***) returns the value of the element stored in the cell of an index specified by the value of the parameter.
  - The second (named ***`Put`***) is able to set a new value of the selected cell (the parameters of the functions specify the index and the value respectively).
- Yes, we agree, both these methods are extremely reckless: **they don't check the index value in any way**.

Arrays of pointers to objects (1)

```
class ArrayList {
        int *values;
        int  size;
public:
        ArrayList(int siz) {
                size = siz;        values = new int[size];
                cout << "Array of " << size << " ints constructed." << endl;
        }
        ~ArrayList(void) {
                delete [] values;
                cout << "Array of " << size << " ints destructed." << endl;
        }
        int Get(int ix) { return values[ix]; }
        void Put(int ix, int val) { values[ix] = val; }
};
int main(void) {
        ArrayList *arr = new Array(2);
        for(int i = 0; i < 2; i++)
                arr->Put(i, i + 100);
        for(int i = 0; i < 2; i++)
                cout << "#" << i + 1 << ":" << arr->Get(i) << endl;
        delete arr;     }
```

Output:

```
Array of 2 ints constructed.
#1:100
#2:101
Array of 2 ints destructed.
```

Arrays of pointers to objects (2)

- There are no obstacles to gathering pointers to objects inside an array.
- We've used two objects of the *Array* class and stored pointers to them in the *arr* array (don't confuse these two entities: *Array* is a class defined by us, *arr* is an ordinary, language built-in array).
- Now carefully follow the flow of the program and note the syntax we've used to access each of the existing objects.

Arrays of pointers to objects (2)

```
class ArrayList {
        int *values;
        int  size;
public:
        ArrayList(int siz) {
                size = siz; values = new int[size];
                cout << "Array of " << size << " ints constructed." << endl;
        }
        ~ArrayList(void) {
                delete [] values;
                cout << "Array of " << size << " ints destructed." << endl;
        }
        int Get(int ix) { return values[ix]; }
        void Put(int ix, int val) { values[ix] = val; }
};
int main(void) {
        ArrayList *arr[2] = { new ArrayList(2), new ArrayList(2) };
        for(int i = 0; i < 2; i++)
            for(int j = 0; j < 2; j++)
                arr[i]->Put(j, j + 10 + i);
        for(int i = 0; i < 2; i++) {
              for(int j = 0; j < 2; j++)
                    cout << "#" << i + 1 << ":" << arr[i]->Get(j) ;
                    cout << endl;
        }
        delete arr[0];  delete arr[1];   }
```

Output:

```
Array of 2 ints constructed.
Array of 2 ints constructed.
#1:10#1:11
#2:11#2:12
Array of 2 ints destructed.
Array of 2 ints destructed.
```